

Acoplamiento en el diseño de software

El acoplamiento es un concepto central del diseño de software porque indica hasta qué punto unas partes del sistema dependen de otras. Cuando esas dependencias son demasiado fuertes, un cambio local puede provocar efectos imprevistos en distintos componentes, lo que complica el mantenimiento y la evolución del sistema.

Hablar de acoplamiento implica hablar también de complejidad. La complejidad esencial pertenece al problema que se intenta resolver, pero la complejidad accidental aparece cuando el sistema se diseña de forma deficiente. Por eso, una parte importante del trabajo arquitectónico consiste en reducir dependencias innecesarias y construir soluciones más claras y predecibles.

Complejidad y modularidad

Un sistema bien diseñado debería responder a los cambios de la forma esperada. Cuando eso no sucede, el software se vuelve difícil de comprender y modificar. En los sistemas muy complejos, anticipar las consecuencias de un cambio deja de ser evidente y obliga a comprobar sus efectos casi mediante prueba y error.

Frente a ese problema, la modularidad actúa como el enfoque opuesto al acoplamiento excesivo. Dividir el sistema en módulos permite localizar mejor dónde debe realizarse una modificación y prever con más facilidad qué partes se verán afectadas. Esto mejora la mantenibilidad y hace más sencilla la evolución del software con el tiempo.

Sin embargo, no basta con separar un sistema en paquetes o microservicios para que sea realmente modular. Cada parte debe tener una responsabilidad clara y relacionarse con las demás de forma controlada. Si eso no ocurre, la separación es solo aparente y el acoplamiento sigue existiendo.

Qué es un módulo

Un módulo puede entenderse como una unidad del sistema que cumple tres criterios. El primero es tener una funcionalidad bien definida. Su propósito debe poder describirse con claridad sin necesidad de examinar su implementación interna. Es decir, debe ser fácil entender qué hace, aunque no se conozca cómo lo hace.

El segundo criterio es disponer de una interfaz pública clara. Otros módulos deben poder usar su funcionalidad sin depender de sus detalles internos. Esa interfaz puede adoptar formas distintas según el nivel de abstracción, como una API REST, una clase pública o la firma de un método. La idea es que el módulo funcione como una caja negra.

El tercer criterio es conservar cierto grado de independencia al modificarse o evolucionar. Si cambiar un módulo obliga a tocar continuamente otros componentes, existe un acoplamiento fuerte. Un ejemplo habitual es el de dos microservicios que comparten directamente la misma base de datos, ya que una modificación del esquema puede afectar a ambos.

Además, los módulos pueden aparecer en distintos niveles del sistema: un microservicio, un paquete, una clase o un método. Lo importante no es su tamaño, sino que cumplan estos principios y ayuden a reducir la complejidad general.

Historia y niveles del acoplamiento

El concepto de acoplamiento surge en los años setenta, dentro del desarrollo de software estructurado, cuando los sistemas comienzan a crecer en tamaño y complejidad. En ese contexto se hace necesario no solo diseñar bien cada parte, sino también comprender cómo se relacionan entre sí los distintos módulos.

El modelo clásico distingue seis niveles de acoplamiento: *Content*, *Common*, *Control*, *Stamp*, *Data* y *Functional*. Estos niveles van desde formas muy problemáticas hasta formas más deseables de relación entre módulos. En los niveles más altos, un módulo conoce detalles internos de otro, comparte datos globales o condiciona su comportamiento. En los niveles más bajos, la comunicación se limita a datos necesarios o a interfaces claras y bien definidas.

La idea principal es que cuanto mayor es el conocimiento que un módulo tiene sobre otro, mayor es el acoplamiento y más frágil se vuelve el sistema. Por eso, los enfoques modernos intentan reducir estas dependencias y favorecer relaciones basadas en contratos públicos estables.

Dimensiones del acoplamiento

El acoplamiento también puede analizarse desde tres dimensiones. La primera es la fuerza de integración, que mide cuánto conoce un módulo a otro. La segunda es el espacio, entendido como la distancia lógica entre módulos y la complejidad de su relación. La tercera es el tiempo, que refleja la frecuencia y la sincronización con la que cambian las distintas partes del sistema.

Estas dimensiones muestran que el acoplamiento no es un problema único, sino un conjunto de factores que afectan a la complejidad global del software. Por ello, el objetivo no es eliminar toda relación entre módulos, sino limitarla a interfaces públicas claras, estables y fáciles de mantener.

Realizado por Hugo Suárez (uo301609), Pablo Hevia (301496) y Mateo Rubio (300069).